Jump Search and Fibonacci Search

Data Structures Using C – Searching Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recap: Searching Algorithms

Algorithm	Requirement	Worst Time
Linear Search	No sorting	$O(n)$
Binary Search	Sorted array	$O(\log n)$
Jump Search	Sorted array	$O(\sqrt{n})$
Fibonacci Search	Sorted array	$O(\log n)$

Today's Focus

Jump Search uses fixed-size jumps followed by a linear scan. Fibonacci Search uses Fibonacci numbers to narrow the search interval.

What is Jump Search?

Jump Search is also called **Block Search**.

- It works on a sorted collection.
- It does not compare every element initially.
- It skips elements using fixed-length jumps.
- It identifies a block that may contain the key.
- It then performs Linear Search inside that block.

$$\boxed{\text{Jump} \rightarrow \text{Find Block} \rightarrow \text{Linear Scan}}$$

Why Do We Need Jump Search?

Consider a sorted array of 1,000 elements.
If the required element occurs near the end:

- Linear Search may perform hundreds of comparisons.
- Jump Search skips several elements at once.
- Only the identified block is scanned sequentially.

Key Idea

Instead of inspecting every element, inspect selected block-ending elements and scan only the relevant block.

Prerequisite: Sorted Array

Consider:

$$A = [1, 2, 3, 4, 5, 6, 7, 8, 9]$$

The values are arranged in ascending order.

Suppose:

$$VAL = 8$$

With a jump size of 3, we inspect block endpoints at indices:

$$2, \quad 5, \quad 8$$

Their values are:

$$3, \quad 6, \quad 9$$

How Does Jump Search Work?

1. Choose a jump length k.
2. Inspect the last element of the current block.
3. If the endpoint is smaller than the key, jump to the next block.
4. When the endpoint is greater than or equal to the key, stop jumping.
5. Perform Linear Search within that block.

If every block endpoint is smaller than the key, the search is unsuccessful.

Textbook Example: Search for 8

Array:

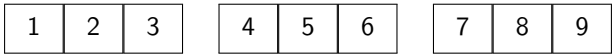
$$A = [1, 2, 3, 4, 5, 6, 7, 8, 9]$$

Number of elements:

$$n = 9$$

Jump length:

$$k = \sqrt{9} = 3$$



Blocks: $[1, 2, 3]$, $[4, 5, 6]$, $[7, 8, 9]$.

Jump Search: First Block

First block:

$$[1, 2, 3]$$

Inspect its endpoint:

$$A[2] = 3$$

Compare:

$$3 < 8$$

The target cannot be in this block.

Decision

Jump to the next block.

Jump Search: Second Block

Second block:

$$[4, 5, 6]$$

Inspect its endpoint:

$$A[5] = 6$$

Compare:

$$6 < 8$$

The target cannot be in this block.

Decision

Jump forward by another block.

Jump Search: Third Block

Third block:

$$[7, 8, 9]$$

Inspect its endpoint:

$$A[8] = 9$$

Compare:

$$9 > 8$$

Since the array is sorted, the target, if present, must lie in the current block.

$$\boxed{\text{Candidate Block} = [7, 8, 9]}$$

Linear Search Within the Block

Candidate block:

$$[7, 8, 9]$$

Check:

$$A[6] = 7 \neq 8$$

Next:

$$A[7] = 8$$

Therefore:

$$\boxed{POS = 7}$$

The element is at index 7, or position 8 using one-based numbering.

Complete Jump Search Trace

Stage	Index	Value	Result
Jump 1	2	3	Continue
Jump 2	5	6	Continue
Jump 3	8	9	Block identified
Scan 1	6	7	No match
Scan 2	7	8	Found

Five key comparisons are shown in this trace.
The exact count depends on implementation details, including whether an endpoint is checked again.

Unsuccessful Search Example

Consider:

$$A = [2, 4, 6, 8, 10, 12, 14, 16, 18]$$

Search:

$$VAL = 11, \qquad k = 3$$

Inspect block endpoints:

$$6 < 11$$

$$12 > 11$$

Candidate block:

$$[8, 10, 12]$$

Scan:

$$8 \neq 11, \quad 10 \neq 11$$

Since $12 > 11$, no match is possible afterward.

$$\boxed{POS = -1}$$

Jump Search: Algorithm

JUMP_SEARCH(A, N, VAL)

1. If $N = 0$, return -1.
2. Set $STEP = \lfloor \sqrt{N} \rfloor$, ensuring $STEP \geq 1$.
3. Set $LOW = 0$ and $HIGH = STEP$.
4. While $LOW < N$ and $A[\min(HIGH, N) - 1] < VAL$:
    1. Set $LOW = HIGH$.
    2. Set $HIGH = HIGH + STEP$.
5. For each valid index i from LOW to $\min(HIGH, N) - 1$:
    1. If $A[i] = VAL$, return i.
    2. If $A[i] > VAL$, stop.
6. Return -1.

Important Variables

Variable	Purpose
N	Number of elements
VAL	Search key
$STEP$	Jump or block length
LOW	Beginning of candidate block
$HIGH$	Exclusive end of block
i	Linear scanning index

The endpoint inspected during jumping is:

$$A[\min(HIGH, N) - 1]$$

This prevents accessing beyond the array.

Jump Search: C Function

```c
#include <math.h>

int jumpSearch(const int a[], int n, int key)
{
    if (n <= 0)
        return -1;

    int step = (int)sqrt((double)n);
    if (step < 1)
        step = 1;

    int low = 0;
    int high = step;

    while (low < n)
    {
        int end = high < n ? high : n;

        if (a[end - 1] >= key)
            break;

        low = high;
        high += step;
    }

    int end = high < n ? high : n;
```

Jump Search: Complete C Program

```c
#include <stdio.h>
#include <math.h>

int jumpSearch(const int a[], int n, int key)
{
    if (n <= 0) return -1;

    int step = (int)sqrt((double)n);
    if (step < 1) step = 1;

    int low = 0, high = step;

    while (low < n)
    {
        int end = high < n ? high : n;
        if (a[end - 1] >= key) break;
        low = high;
        high += step;
    }

    int end = high < n ? high : n;

    for (int i = low; i < end; i++)
    {
        if (a[i] == key) return i;
        if (a[i] > key) break;
```

Program Output and Compilation

Expected Output

```
Found at index 7
```

Compile with GCC:

```
gcc jump_search.c -o jump_search -lm
```

The `-lm` option links the math library on systems that require it for `sqrt()`.

How Do We Choose the Jump Length?

Let:

$$n = \text{array size}$$

$$k = \text{jump length}$$

Approximately:

- n/k jumps may be required.
- Up to k elements may be scanned in the identified block.

Therefore:

$$T(n, k) = O\left(\frac{n}{k} + k\right)$$

Deriving the Optimal Jump Length

We want to minimize:

$$f(k) = \frac{n}{k} + k$$

Differentiate:

$$f'(k) = -\frac{n}{k^2} + 1$$

Set:

$$f'(k) = 0$$

Then:

$$\frac{n}{k^2} = 1$$

Therefore:

$$\boxed{k = \sqrt{n}}$$

This is the standard asymptotically optimal fixed jump length.

Effect of Jump Length

Suppose:

$$n = 100$$

Jump length	Approx. jumps	Block scan
1	100	1
5	20	5
10	10	10
20	5	20
50	2	50

The balance is near:

$$\sqrt{100} = 10$$

Jump Search: Complexity

With:

$$k = \sqrt{n}$$

We obtain:

$$T(n) = O\left(\frac{n}{\sqrt{n}} + \sqrt{n}\right)$$

Simplifying:

$$T(n) = O(\sqrt{n} + \sqrt{n})$$

Hence:

$$\boxed{T(n) = O(\sqrt{n})}$$

The worst-case time is:

$$\boxed{\Theta(\sqrt{n})}$$

Jump Search: Complexity Summary

Case	Complexity
Best	$\Theta(1)$
Average, typical uniform model	$\Theta(\sqrt{n})$
Worst	$\Theta(\sqrt{n})$
Auxiliary space	$O(1)$

Important

The best case occurs when the key is found at the first inspected block endpoint or during the first scan comparison.

Example: 1,000 Elements

Suppose:

$$A = [0, 1, 2, \ldots, 999]$$

Search:

$$VAL = 674$$

Choose:

$$k = \lfloor \sqrt{1000} \rfloor = 31$$

Block endpoints include:

$$30, 61, 92, \ldots, 650, 681$$

The key lies in:

$$[651, \ldots, 681]$$

Jump Search requires far fewer comparisons than a sequential scan from index 0.

Jump Search vs Linear Search

Property	Linear	Jump
Sorted input required	No	Yes
Basic technique	Scan	Jump + scan
Worst time	$O(n)$	$O(\sqrt{n})$
Extra space	$O(1)$	$O(1)$

Jump Search reduces comparisons on large sorted arrays by skipping blocks.

Jump Search vs Binary Search

Property	Jump	Binary
Sorted input	Required	Required
Worst time	$O(\sqrt{n})$	$O(\log n)$
Search strategy	Blocks	Halving
Extra space	$O(1)$	$O(1)$

Important Clarification

Binary Search has a better asymptotic comparison bound. Jump Search may be useful when the cost of access or movement makes short forward jumps preferable, but it is not universally faster than Binary Search.

Further Optimization: Repeated Jumping

The textbook discusses applying a smaller jump search within an identified block.
Example:

$$n = 1,000,000$$

Initial step:

$$\sqrt{1,000,000} = 1000$$

The selected block may contain 1000 elements.
A second jump length could be:

$$\sqrt{1000} \approx 31$$

This is a multi-level variation, not the standard single-level Jump Search. Its precise complexity depends on how the levels and their costs are defined.

Advantages of Jump Search

- Simple concept.
- Fewer comparisons than Linear Search on many large sorted arrays.
- Uses constant auxiliary space.
- Easy to implement iteratively.
- Can be adapted to block-based searching environments.

Limitations of Jump Search

- Requires sorted data.
- Asymptotically slower than Binary Search on random-access arrays.
- Performance depends on jump length.
- The identified block still requires scanning.
- Sorting an unsorted input adds preprocessing cost.

Introduction to Fibonacci Search

Fibonacci Search is another searching technique for sorted arrays.
It uses Fibonacci numbers to choose search positions.

Fibonacci sequence:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

Each term is:

$$F_k = F_{k-1} + F_{k-2}$$

The textbook attributes the method to Kiefer (1953).

How Does Fibonacci Search Work?

- Start with Fibonacci numbers large enough to cover the array length.
- Choose a probe position using a Fibonacci offset.
- Compare the probe element with the key.
- Reduce the search range using smaller Fibonacci numbers.
- Continue until found or exhausted.

Fibonacci-Guided Interval Reduction

Fibonacci Search vs Binary Search

Property	Binary	Fibonacci
Sorted array	Yes	Yes
Interval reduction	Near halves	Fibonacci ratios
Worst time	$O(\log n)$	$O(\log n)$
Iterative space	$O(1)$	$O(1)$

Unlike Binary Search, Fibonacci Search does not always probe the middle index.

Fibonacci Search: Worked Example

Consider:

$$A = [10, 20, 30, 40, 50, 60, 70, 80, 90]$$

Search:

$$VAL = 70$$

Number of elements:

$$n = 9$$

Smallest Fibonacci number not less than 9:

$$13$$

Associated Fibonacci values:

$$F_m = 13, \quad F_{m-1} = 8, \quad F_{m-2} = 5$$

Fibonacci Search: First Probe

Initially:

$$offset = -1$$

Probe index:

$$i = \min(offset + F_{m-2}, n - 1)$$

$$i = \min(-1 + 5, 8) = 4$$

Compare:

$$A[4] = 50$$

Since:

$$50 < 70$$

Discard the prefix through index 4.
Set:

$$offset = 4$$

Fibonacci Search: Second Probe

After reducing the Fibonacci numbers:

$$F_m = 8, \quad F_{m-1} = 5, \quad F_{m-2} = 3$$

Probe:

$$i = \min(4 + 3, 8) = 7$$

Compare:

$$A[7] = 80$$

Since:

$$80 > 70$$

Continue in the remaining left segment.

Fibonacci Search: Third Probe

After the next reduction:

$$F_m = 3, \quad F_{m-1} = 2, \quad F_{m-2} = 1$$

Probe:

$$i = \min(4 + 1, 8) = 5$$

Compare:

$$A[5] = 60 < 70$$

Update:

$$offset = 5$$

The remaining candidate is index 6, which contains:

$$A[6] = 70$$

Fibonacci Search: Final Result

The next probe reaches:

$$i = 6$$

Compare:

$$A[6] = 70$$

Therefore:

$$\boxed{POS = 6}$$

Fibonacci Search has found the element using Fibonacci-guided probe positions.

Fibonacci Search: C Function, Part 1

```
1  int fibonacciSearch(const int a[], int n, int key)
2  {
3      int fib2 = 0;
4      int fib1 = 1;
5      int fib = fib1 + fib2;
6
7      while (fib < n)
8      {
9          fib2 = fib1;
10         fib1 = fib;
11         fib = fib1 + fib2;
12     }
13
14     int offset = -1;
15
16     while (fib > 1)
17     {
18         int i = offset + fib2;
19         if (i >= n)
20             i = n - 1;
```

Fibonacci Search: C Function, Part 2

```
        if (a[i] < key)
        {
            fib = fib1;
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
        }
        else if (a[i] > key)
        {
            fib = fib2;
            fib1 = fib1 - fib2;
            fib2 = fib - fib1;
        }
        else
        {
            return i;
        }
    }

    if (fib1 && offset + 1 < n &&
        a[offset + 1] == key)
        return offset + 1;

    return -1;
}
```

Fibonacci Search: Main Program

```c
#include <stdio.h>

/* Place fibonacciSearch() above main(). */

int main(void)
{
    int a[] = {
        10,20,30,40,50,60,70,80,90
    };

    int n = sizeof(a) / sizeof(a[0]);
    int key = 70;

    int pos = fibonacciSearch(a, n, key);

    if (pos == -1)
        printf("Element not found\n");
    else
        printf("Found at index %d\n", pos);

    return 0;
}
```

Expected output:

```
Found at index 6
```

Fibonacci Search: Complexity

Fibonacci numbers grow exponentially with their sequence index.
Consequently, only logarithmically many interval reductions are needed.

$$\boxed{T(n) = O(\log n)}$$

For the iterative implementation:

$$\boxed{\text{Auxiliary Space} = O(1)}$$

The array must already be sorted.

Searching Algorithms: Comparison

Property	Linear	Binary	Jump	Fibonacci
Sorted input	No	Yes	Yes	Yes
Best case	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(\log n)$	$O(\sqrt{n})$	$O(\log n)$
Iterative space	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Observation

For a sorted random-access array, Binary and Fibonacci Search offer better asymptotic comparison bounds than standard Jump Search.

Which Algorithm Should Be Used?

- **Linear Search:** Small or unsorted collections.
- **Binary Search:** Sorted arrays with efficient random access.
- **Jump Search:** Sorted data where block-based scanning or access patterns make jumps useful.
- **Fibonacci Search:** Sorted arrays where Fibonacci-based probing is appropriate.

Activity 1: Jump Search

Consider:

$$A = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]$$

Search:

$$VAL = 18$$

Use:

$$k = \lfloor \sqrt{12} \rfloor = 3$$

Tasks:

1. Identify all block endpoints inspected.
2. Find the candidate block.
3. Perform the linear scan.
4. State the resulting index.

Activity 1: Solution

Block endpoints inspected:

$$A[2] = 6$$

$$A[5] = 12$$

$$A[8] = 18$$

The candidate block is:

$$[14, 16, 18]$$

Linear scan:

$$14 \neq 18, \quad 16 \neq 18, \quad 18 = 18$$

Result:

$$\boxed{POS = 8}$$

Activity 2: Step Length

For:

$$n = 400$$

Find the recommended jump length.

Activity 2: Step Length

For:

$$n = 400$$

Find the recommended jump length.

$$k = \sqrt{400} = 20$$

Approximately 20 jumps and up to 20 block-scan comparisons may be needed.

$$\boxed{O(\sqrt{400}) = O(20)}$$

MCQ 1

Jump Search requires:

1. An unsorted array
2. A sorted array
3. A binary tree
4. A hash table

MCQ 1

Jump Search requires:

1. An unsorted array
2. A sorted array
3. A binary tree
4. A hash table

Answer

Option 2: A sorted array.

MCQ 2

The usual jump length for n elements is:

1. n
2. $\log_2 n$
3. $\sqrt{n}$
4. n^2

MCQ 2

The usual jump length for n elements is:

1. n
2. $\log_2 n$
3. $\sqrt{n}$
4. n^2

Answer

Option 3: $\sqrt{n}$.

MCQ 3

After finding the candidate block, Jump Search normally performs:

1. Heap Sort
2. Linear Search
3. Merge Sort
4. Hashing

MCQ 3

After finding the candidate block, Jump Search normally performs:

1. Heap Sort
2. Linear Search
3. Merge Sort
4. Hashing

Answer

Option 2: Linear Search.

MCQ 4

The worst-case complexity of standard Jump Search with optimal fixed jumps is:

1. $O(1)$
2. $O(\sqrt{n})$
3. $O(n^2)$
4. $O(n \log n)$

MCQ 4

The worst-case complexity of standard Jump Search with optimal fixed jumps is:

1. $O(1)$
2. $O(\sqrt{n})$
3. $O(n^2)$
4. $O(n \log n)$

Answer

Option 2: $O(\sqrt{n})$.

MCQ 5

Fibonacci Search uses:

1. Random pivots
2. Fibonacci numbers
3. Hash functions
4. Heap priorities

MCQ 5

Fibonacci Search uses:

1. Random pivots
2. Fibonacci numbers
3. Hash functions
4. Heap priorities

Answer

Option 2: Fibonacci numbers.

Practice Questions

1. Define Jump Search.
2. Explain the working of Jump Search with a suitable example.
3. Why must the array be sorted?
4. Derive the optimal jump length.
5. Write the Jump Search algorithm.
6. Implement Jump Search in C.
7. Compare Jump Search and Binary Search.
8. Explain Fibonacci Search.
9. State the complexity of Fibonacci Search.

Lecture Summary

- Jump Search is a block-based searching technique.
- It requires sorted input.
- Its usual step length is $\sqrt{n}$.
- It performs a linear scan inside the identified block.
- Worst-case Jump Search time: $O(\sqrt{n})$.
- Fibonacci Search uses Fibonacci-guided probes.
- Fibonacci Search time: $O(\log n)$.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 14: Searching and Sorting, Jump Search and Fibonacci Search.
2. Thomas H. Cormen et al., *Introduction to Algorithms*.
3. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.

Lecture Feedback

Your Feedback Matters

Please share feedback on:

- Jump Search concepts
- Worked examples
- Step-size derivation
- C implementation
- Fibonacci Search
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Jump Search & Fibonacci Search